

h.Sum(data) Does Not Hash Data



Ruslan Kadyrov, Mercari Inc., Japan

GopherCon 2026

1. Naive assumption

```
8 ▶ func main() {  
9     h := sha256.New()  
10    data1 := []byte("hello world :)")  
11    hash1 := h.Sum(data1)  
12    fmt.Printf(format: "%x", hash1)  
13 }
```

```
68656c6c6f20776f726c64203a29e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855
```

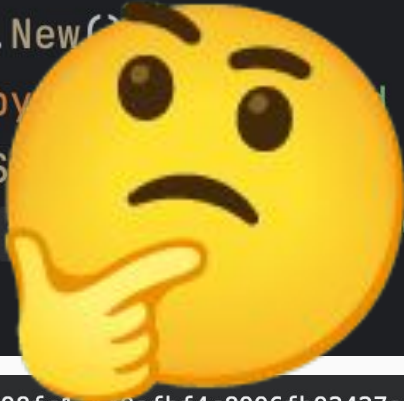
2. What's wrong with the length of the hash?

```
8  ▶ func main() {  
9      h := sha256.New()  
10     data1 := []byte("hello world :)")  
11     hash1 := h.Sum(data1)  
12     fmt.Printf(format: "%x - %d\n", hash1, len(hash1))  
13 }
```

68656c6c6f20776f726c64203a29e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855 - 46

2. What's wrong with the length of the hash?

```
8 ▶ func main() {  
9     h := sha256.New()  
10    data1 := []byte("data1 :)")  
11    hash1 := h.Sum(data1)  
12    fmt.Printf("hash1: %x", hash1, len(hash1))  
13 }
```

A large yellow thinking face emoji is overlaid on the code, specifically covering the line where the hash is calculated and printed. The emoji has a hand on its chin and a questioning expression, suggesting a problem or a point for reflection.

68656c6c6f20776f726c64203a29e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855 - 46

3. Trying two different inputs

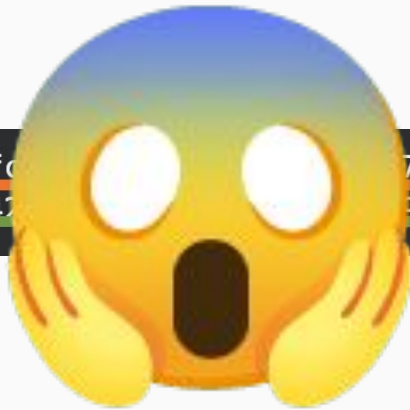
```
8  ▶ func main() {  
9      h := sha256.New()  
10     data1 := []byte("hello world :)")  
11     hash1 := h.Sum(data1)  
12     fmt.Printf(format: "%x - %d\n", hash1, len(hash1))  
13  
14     data2 := []byte("goodbye world :(")  
15     hash2 := h.Sum(data2)  
16     fmt.Printf(format: "%x - %d\n", hash2, len(hash2))  
17 }
```

```
68656c6c6f20776f726c64203a29e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855 - 46  
676f6f6462796520776f726c64203a28e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855 - 48
```

4. Hashing output analysis

```
68656c6c6f20776f726c64203a29e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855 - 46  
676f6f6462796520776f726c64203a28e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855 - 48
```

4. Hashing output analysis



```
68656c6c6f20776f726c64203a29e3b0c44298f07ae41e4649b934ca495991b7852b855 - 46  
676f6f6462796520776f726c64203a28e3b0c44298f07ae41e4649b934ca495991b7852b855 - 48
```

5. Two supported hashing APIs

1. Instant hash

```
data := []byte("Hello")  
sum := sha256.Sum256(data)
```

2. Incremental/running/streaming hash

```
h := sha256.New()  
h.Write([]byte("Hel"))  
h.Write([]byte("lo"))  
sum := h.Sum(b: nil)
```

6. What's the need for streaming hash?

Why?

Streaming hasher doesn't keep the whole input in memory. It only keeps a small working buffer and the current hash state.

Golang streaming/running hashers process data incrementally using the standard `hash.Hash` interface, enabling efficient chunk-by-chunk computation.

When?



Large Files



Network



io.Reader

7. Moment of truth

type Hash ¶

```
type Hash interface {  
    // Write (via the embedded io.Writer interface) adds more data to the running hash.  
    // It never returns an error.  
    io.Writer  
  
    // Sum appends the current hash to b and returns the resulting slice.  
    // It does not change the underlying hash state.  
    Sum(b []byte) []byte
```

8. Misuse of interface API

```
8 ▶ func main() {  
9     h := sha256.New()  
10    data1 := []byte("hello world :)")  
11    // ⚠ No writes to hasher  
12    // Supposed to be h.Write(data1)  
13    hash1 := h.Sum(data1)  
14    // ⚠ Since we don't need raw data to be prepended to the hash,  
15    // we should pass nil instead of data1  
16    fmt.Printf(format: "%x - %d\n", hash1, len(hash1))  
17 }
```

9. Hashing output explanation

```
68656c6c6f20776f726c64203a29e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855 - 46
676f6f6462796520776f726c64203a28e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855 - 48
```

Raw argument
of sum()

SHA256 hash of an empty
message in hasher's working
buffer (32 bytes)

10. Key takeaways

1. Having all the data already in memory?
Use `sha256.Sum256()`.
2. Processing big amount of data incrementally?
Use `hash.Hash: Write() → Sum(nil)`.
3. Remember: `h.Sum(data)` does not hash `data`.

Read the docs.

Thanks!

Feel free to contact me!

My LinkedIn page



My (upcoming) blog

